

CHAPTER 8

組合邏輯電路與 簡易的算術邏輯運算

Millions

在本章將為各位介紹：組合邏輯 (Combination Logic) 電路，以及簡易的算術邏輯運算單元 (ALU) 的設計…等等的觀念，作一個簡要的說明。



8.1 組合邏輯 (Combination Logic) 電路

組合邏輯 (Combination Logic) 電路與循序邏輯 (Sequential Logic) 電路為數位系統的二大分類，而循序邏輯電路又或多或少的需要配合組合邏輯電路才能設計出想要的電路來，因此組合邏輯電路是數位電路中最基礎也是最重要的部份。

8.1.1 布林方程式之實作 (Implementation of Boolean Equation)

以往使用傳統的設計方法來實作布林方程式，首先要從布林方程式的化簡開始，在化簡的過程中經常需要一些繁瑣的步驟 (例如：布林代數化簡、卡諾圖化簡...等等的化簡方法)，使得設計的過程既耗時又困難。

相反地，使用 Verilog 硬體描述語言來描述布林方程式的設計，我們只需要將此布林方程式以邏輯操作來表示，Verilog 合成器 (Synthesizer) 在執行最佳化 (Optimization) 的過程，會依據您對合成器設定的 Verilog 編譯命令 (Compiler Directives) 來化簡該邏輯電路。

通常我們可以設定合成器化簡的準則，如下：

- 以電路的面積 (Area) 為最高優先考量。
- 以電路的運作速度 (Speed) 為最高優先考量。
- 以電路的電源消耗 (Power Consumption) 為最高優先考量。

- 可以同時混合以上的準則 (同時考慮二個或三個) 讓合成器作為電路合成的考量因素。

在下面例子中，我們用 Verilog 硬體描述語言來描述下列的布林方程式。

```
X = A · C · D' + B · C' + B · D + C · D ;
Y = A' + B + C ;
```

這 2 個布林方程式，分別描述了該邏輯電路中的 2 個輸出變數 X 及 Y。其係由 4 個變數 A、B、C 及 D 作為布林方程式的輸入變數。

$X = A \cdot C \cdot D' + B \cdot C' + B \cdot D + C \cdot D$; 及 $Y = A' + B + C$; 的真值表

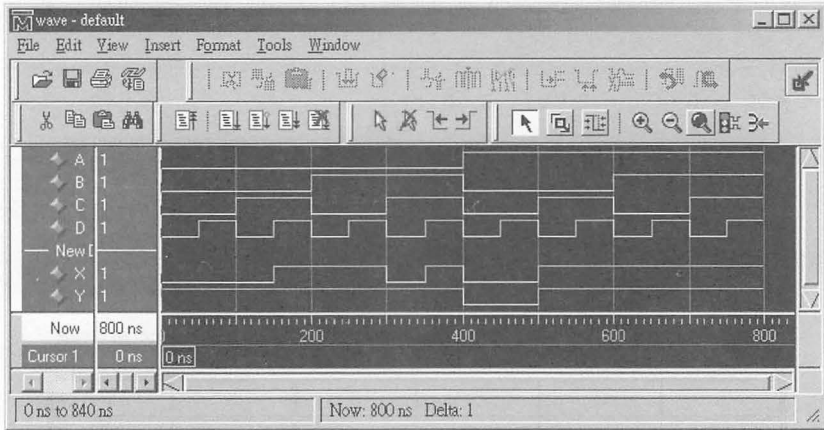
A	B	C	D	X	Y
0	0	0	0	0	1
0	0	0	1	0	1
0	0	1	0	0	1
0	0	1	1	1	1
0	1	0	0	1	1
0	1	0	1	1	1
0	1	1	0	0	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	1	1

```
// Boolean.V: 布林方程式之實現，範例 1
module Boolean (A, B, C, D, X, Y);
input  A, B, C, D;
output X, Y;
wire  X, Y;
```

```

assign X = (A & C & (!D)) | (B & (!C)) | (B & D) | (C & D);
assign Y = (!A) | B | C;
endmodule

```



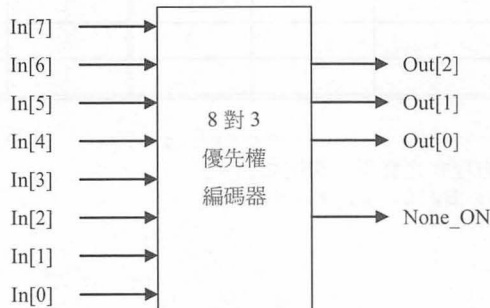
▲ Boolean.V 的模擬結果

8.1.2 編碼器 (Encoder)

一般而言，編碼器相當於具有 OR 的功能，將多個輸入線 OR 成一個輸出線。

在編碼器中，每次只允許一條輸入線啟動，因為如果同時有多條輸入線啟動時，則編碼器的輸出將無法代表任一條輸入線。

例 1 8 對 3 優先權編碼器 (Priority Encoder)

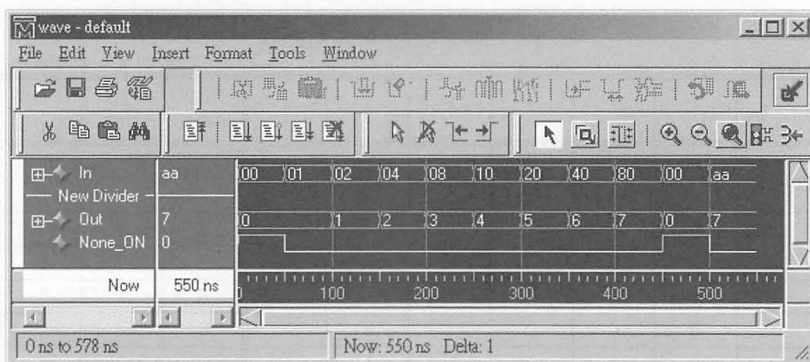


8 對 3 優先權編碼器真值表

In[7]	In[6]	In[5]	In[4]	In[3]	In[2]	In[1]	In[0]	Out [2]	Out [1]	Out [0]	None _ON
1	X	X	X	X	X	X	X	1	1	1	0
0	1	X	X	X	X	X	X	1	1	0	0
0	0	1	X	X	X	X	X	1	0	1	0
0	0	0	1	X	X	X	X	1	0	0	0
0	0	0	0	1	X	X	X	0	1	1	0
0	0	0	0	0	1	X	X	0	1	0	0
0	0	0	0	0	0	1	X	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	1

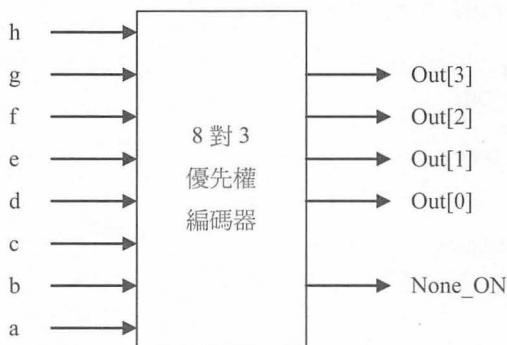
```
// Encoder1.V: 優先權編碼器 (Priority Encoder)
// 傳回 Highest Bit 為 1 的位置
module Encoder1 (In, Out, None_ON);
input  [7:0] In;
output [2:0] Out;
output      None_ON;
reg  [2:0] Out;
reg      None_ON;

always @(In)
begin: Encoder_Block
    integer i
    Out = 0;
    None_ON = 1;
    for (i = 0; i < 8; i = i + 1)
    begin
        if (In[i]) // 傳回 Highest Bit 為 1 的位置
        begin
            Out = i;
            None_ON = 0;
        end
    end
end
endmodule
```



▲ Encoder1.V 的模擬結果

例 2 使用 assign 敘述來處理一個優先權編碼器 (Priority Encoder) 的輸入及輸出訊號。



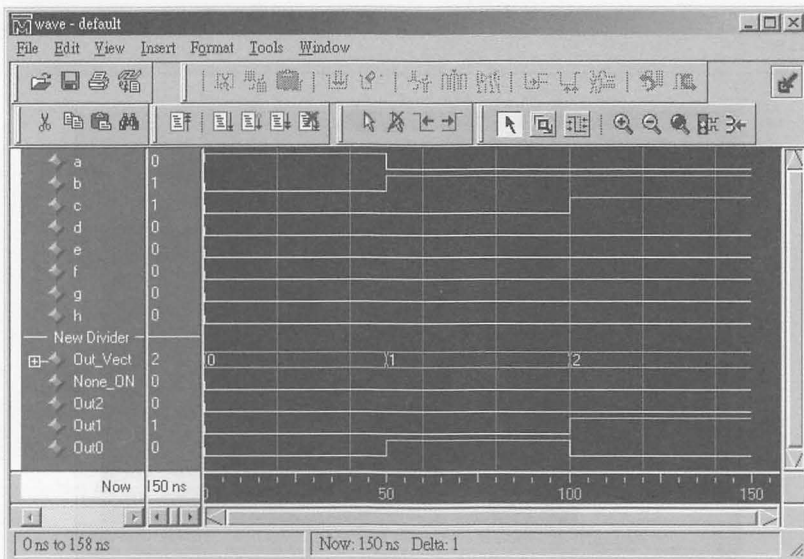
8 對 3 優先權編碼器真值表

h	g	f	e	d	c	b	a	Out[3]	Out[2]	Out[1]	Out[0]	None_ON
1	X	X	X	X	X	X	X	0	1	1	1	0
0	1	X	X	X	X	X	X	0	1	1	0	0
0	0	1	X	X	X	X	X	0	1	0	1	0
0	0	0	1	X	X	X	X	0	1	0	0	0
0	0	0	0	1	X	X	X	0	0	1	1	0
0	0	0	0	0	1	X	X	0	0	1	0	0
0	0	0	0	0	0	1	X	0	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	1

```
// Encoder2.V: 優先權編碼器 (Priority Encoder)
module Encoder2 (a, b, c, d, e, f, g, h, Out_Vect, None_ON, Out2,
Out1, Out0);
input      a, b, c, d, e, f, g, h;
output     [3:0] Out_Vect;
output     None_ON, Out2, Out1, Out0;
wire       [3:0] Out_Vect;
wire       None_ON, Out2, Out1, Out0;

assign Out_Vect = h ? 4'b0111: // 如果 h =1, 則 Out_Vect = 4'b0111
                g ? 4'b0110: // 否則如果 g =1, 則 Out_Vect = 4'b0110
                f ? 4'b0101: // 否則如果 f =1, 則 Out_Vect = 4'b0101
                e ? 4'b0100: // 否則如果 e =1, 則 Out_Vect = 4'b0100
                d ? 4'b0011: // 否則如果 d =1, 則 Out_Vect = 4'b0011
                c ? 4'b0010: // 否則如果 c =1, 則 Out_Vect = 4'b0010
                b ? 4'b0001: // 否則如果 b =1, 則 Out_Vect = 4'b0001
                a ? 4'b0000: // 否則如果 a =1, 則 Out_Vect = 4'b0000
                4'b1000; // 否則 Out_Vect = 4'b1000

assign None_ON = Out_Vect [3];
assign Out2    = Out_Vect [2];
assign Out1    = Out_Vect [1];
assign Out0    = Out_Vect [0];
endmodule
```



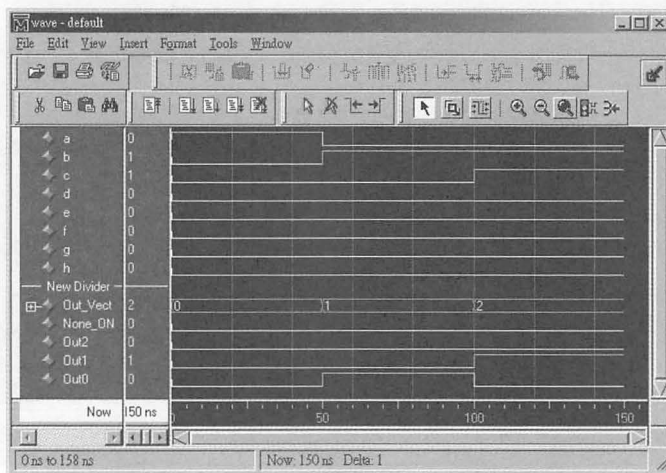
▲ Encoder2.V 的模擬結果

例 3 使用 always 及 if...else...敘述來處理一個優先權編碼器 (Priority Encoder) 的輸入訊號，配合 assign 敘述來處理優先權編碼器的輸出訊號。

```
// Encoder3.V: 優先權編碼器 (Priority Encoder), 範例 3
module Encoder3 (a, b, c, d, e, f, g, h, Out_Vect, None_ON, Out2,
Out1, Out0);
input      a, b, c, d, e, f, g, h;
output [3:0] Out_Vect;
output      None_ON, Out2, Out1, Out0;
reg [3:0] Out_Vect;
wire      None_ON, Out2, Out1, Out0;

assign {None_ON, Out2, Out1, Out0} = Out_Vect;

always @(a or b or c or d or e or f or g or h)
begin
    if (h) Out_Vect = 4'b0111;
    else if (g) Out_Vect = 4'b0110;
    else if (f) Out_Vect = 4'b0101;
    else if (e) Out_Vect = 4'b0100;
    else if (d) Out_Vect = 4'b0011;
    else if (c) Out_Vect = 4'b0010;
    else if (b) Out_Vect = 4'b0001;
    else if (a) Out_Vect = 4'b0000;
    else Out_Vect = 4'b1000;
end
endmodule
```



▲ Encoder3.V 的模擬結果

8.1.3 解碼器 (Decoder)

解碼器在數位系電路的應用上，可以說是非常地普遍，通常被用在將位址匯流排與控制訊號，來分解成很多個用途使用。例如：在 PC 的 I/O 定址中，000000H 到 0003FFH 的 1K 位址中，一般將 0220H 到 022FH 解碼為聲霸卡（知名的音效卡）的輸入／輸出範圍；將 0201H 解碼為 Gameport Joystick 的輸入／輸出範圍；將 0240H 到 025FH 解碼為 D-Link 網路卡的輸入／輸出範圍...等等。

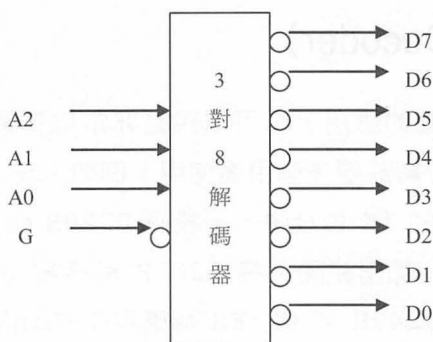
注意 NOTICE

這些介面卡的輸入／輸出範圍都是可以經由該介面卡之驅動程式來設定，再到 Windows 中「控制台」裡的「系統」中之「裝置管理員」來調整，只要所有的介面裝置之輸入／輸出範圍（I/Obase）、中斷要求（IRQ）以及直接記憶體存取（DMA）這三項的設定值都沒有重覆發生衝突的現象，那麼所接上的裝置就可以正常的運作。

例 4 在本章節中，我們使用 Verilog 硬體描述語言來描述一個 3 對 8 的解碼器作為例子。

3 對 8 的解碼器真值表

G	A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0
1	X	X	X	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	0	1	1	1	1
0	1	0	1	1	1	0	1	1	1	1	1
0	1	1	0	1	0	1	1	1	1	1	1
0	1	1	1	0	1	1	1	1	1	1	1



```
// Dec_3to8.V: 3 對 8 的解碼器
module Decoder_3to8 (A2, A1, A0, G, D7, D6, D5, D4, D3, D2, D1, D0);
input  A2, A1, A0, G;
output D7, D6, D5, D4, D3, D2, D1, D0;
reg    D7, D6, D5, D4, D3, D2, D1, D0;

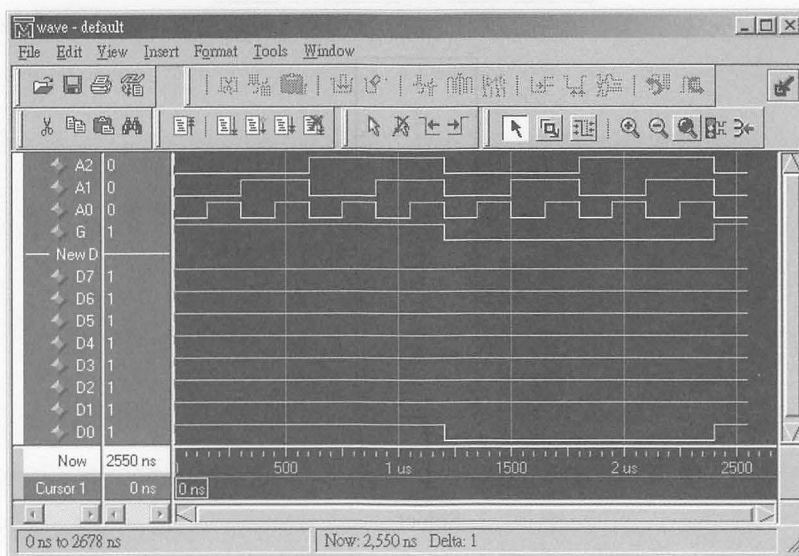
always @(G)
begin
    if (G == 1'b0)
    begin
        case ({A2, A1, A0})
            3'b000:
            begin
                D7 = 1; D6 = 1; D5 = 1; D4 = 1;
                D3 = 1; D2 = 1; D1 = 1; D0 = 0;
            end
            3'b001:
            begin
                D7 = 1; D6 = 1; D5 = 1; D4 = 1;
                D3 = 1; D2 = 1; D1 = 0; D0 = 1;
            end
            3'b010:
            begin
                D7 = 1; D6 = 1; D5 = 1; D4 = 1;
                D3 = 1; D2 = 0; D1 = 1; D0 = 1;
            end
            3'b011:
            begin
                D7 = 1; D6 = 1; D5 = 1; D4 = 1;
                D3 = 0; D2 = 1; D1 = 1; D0 = 1;
            end
            3'b100:
            begin
                D7 = 1; D6 = 1; D5 = 1; D4 = 0;
                D3 = 1; D2 = 1; D1 = 1; D0 = 1;
            end
        endcase
    end
end
```



```

        end
3'b101:
    begin
        D7 = 1; D6 = 1; D5 = 0; D4 = 1;
        D3 = 1; D2 = 1; D1 = 1; D0 = 1;
    end
3'b110:
    begin
        D7 = 1; D6 = 0; D5 = 1; D4 = 1;
        D3 = 1; D2 = 1; D1 = 1; D0 = 1;
    end
3'b111:
    begin
        D7 = 0; D6 = 1; D5 = 1; D4 = 1;
        D3 = 1; D2 = 1; D1 = 1; D0 = 1;
    end
endcase
end
else
begin
    D7 = 1; D6 = 1; D5 = 1; D4 = 1;
    D3 = 1; D2 = 1; D1 = 1; D0 = 1;
end
end
endmodule

```



▲ Dec_3to8.V 的模擬結果

8.1.4 多工器 (Multiplexier)

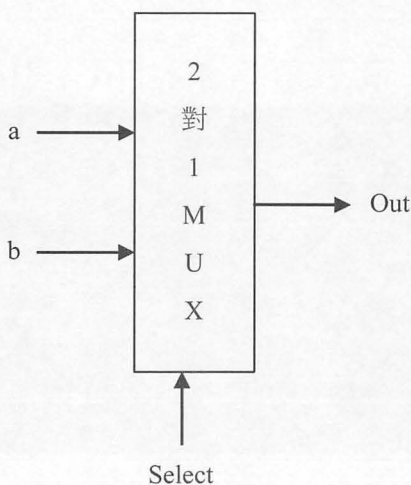
多功器的功能為從多個輸入線中選取一條輸入線，並將其資訊置於單一的輸出線上。又被稱為資料選擇器(Data Selector)。

在本章節中，我們使用 Verilog 硬體描述語言的各種寫法來描述一個 2X1 的多工器、一個 4X1 的多工器，以及使用二個 2X1 的多工器擴充成一個 4X1 的多工器。

(一) 2X1 的多工器

2X1 多工器的真值表

Select	Out
0	a
1	b
X	X

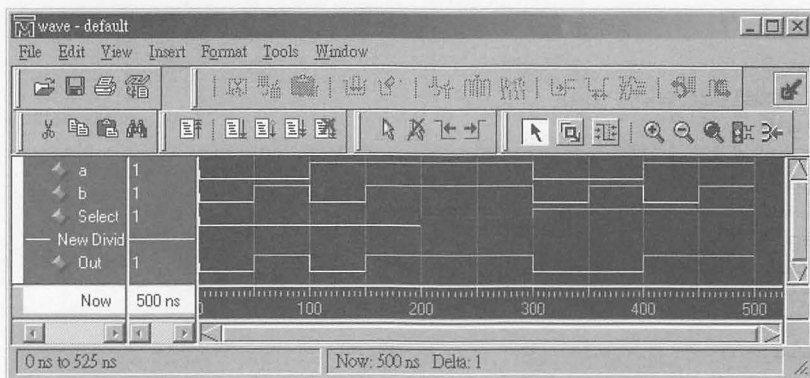


▲ 2X1 的多工器的邏輯符號

例 1 使用 if ... else ... 敘述，描述一個 2 對 1 的 Multiplexier。

```
// Mux2_1_1.V: 2 對 1 的 Multiplexier, 第 1 種寫法
module Mux2to1_1 (a, b, Select, Out);
input  a, b, Select;
output Out;
reg    Out;

always @(a or b or Select)
begin
    if (Select)
        Out = a;
    else
        Out = b;
    end
endmodule
```



▲ Mux2_1_1.V 的模擬結果

例 2 使用 assign 敘述，描述一個 2 對 1 的 Multiplexier。

```
// Mux2_1_2.V: 2 對 1 的 Multiplexier, 第 2 種寫法
module Mux2to1_2 (a, b, Select, Out);
input  a, b, Select;
output Out;
wire   Out;

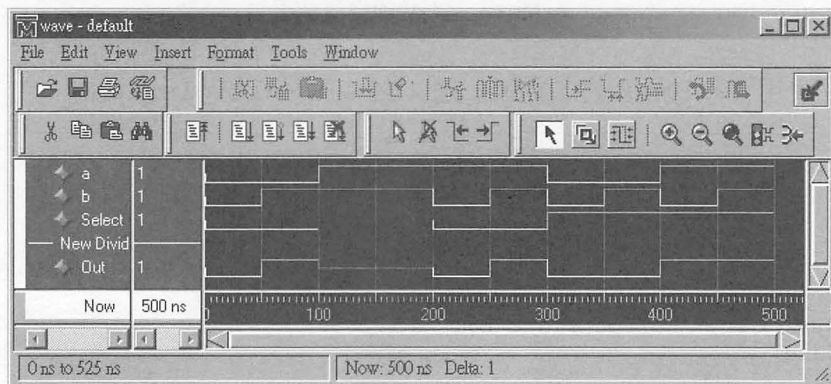
assign Out = Select ? a : b;
endmodule
```

Mux2_1_2.V 的模擬結果和 Mux2_1_1.V 的模擬結果相同。

例 3 使用 if ... else ... 敘述描述一個 2 對 1 的 Multiplexier，且多考慮了：資料選擇線 (Select) 為不確定值 (Unknow) 的處理方式。

```
// Mux2_1_3.V: 2 對 1 的 Multiplexier, 第 3 種寫法
// 這種寫法多考慮了:
// 如果 Select 不等於 1 也不等於 0 時,
// 則令 Out = 1'bx; (輸出值設為 x, 不確定值, Unknow)
module Mux2to1_3 (a, b, Select, Out);
input  a, b, Select;
output Out;
reg    Out;

always @(a or b or Select)
begin
    if (Select == 1'b1)
    begin
        Out = a;
    end
    else
    begin
        if (Select == 1'b0)
        begin
            Out = b;
        end
        else
        begin
            Out = 1'bx; // 不確定值, Unknow
        end
    end
end
endmodule
```

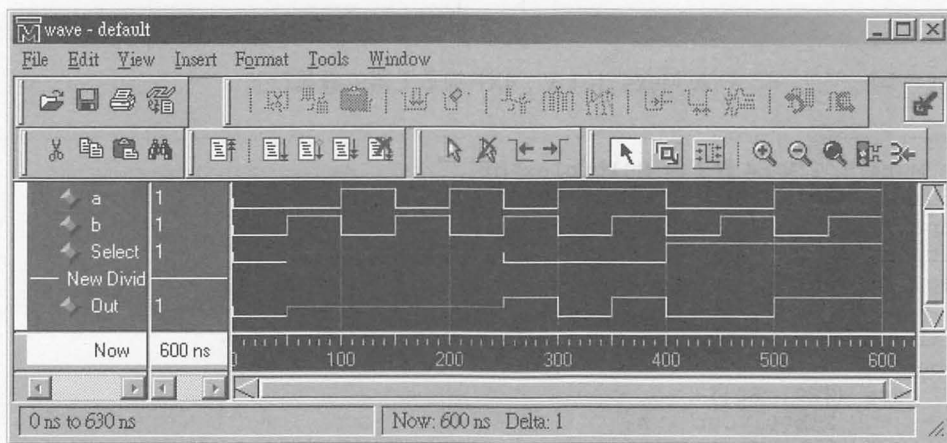


▲ Mux2_1_3.V 的模擬結果

例 4 使用 case 敘述描述一個 2 對 1 的 Multiplexier，且多考慮了：資料選擇線 (Select) 為不確定值 (Unknow) 的處理方式。

```
/ Mux2_1_4.V: 2 對 1 的 Multiplexier, 第 4 種寫法
// 這種寫法多考慮了:
// 如果 Select 不等於 1 也不等於 0 時,
// 則令 Out = 1'bx; (輸出值設為 x, 不確定值, Unknow)
module Mux2to1_4 (a, b, Select, Out);
input    a, b, Select;
output   Out;
reg      Out;

always @(a or b or Select)
begin
    case (Select)
        1'b1: Out = a;
        1'b0: Out = b;
        default: Out = 1'bx; // 不確定值, Unknow
    endcase
end
endmodule
```

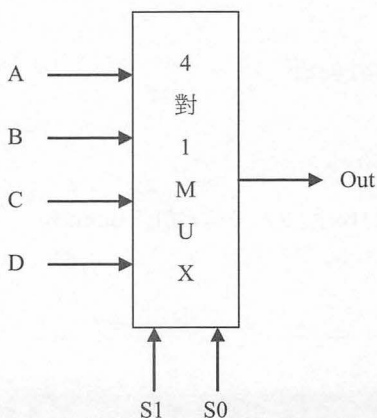


▲ Mux2_1_4.V 的模擬結果

(二) 4X1 的多工器

4X1 多工器的真值表

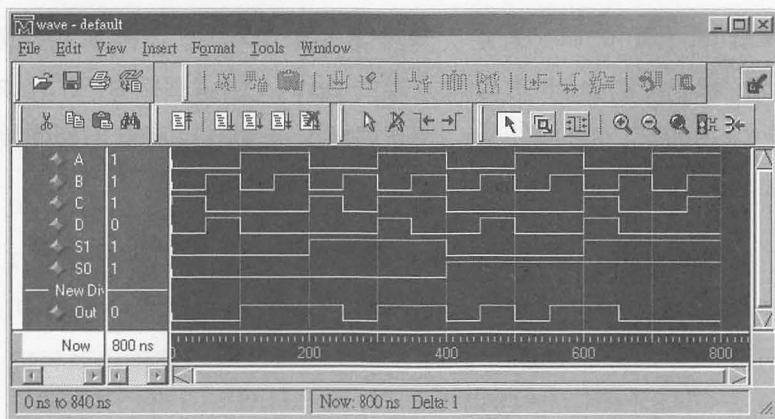
S1	S0	Out
0	0	A
0	1	B
1	0	C
1	1	D



▲ Mux4_1_1.V : 4X1 的多工器的邏輯符號

```
// Mux4_1_1.V : 4 對 1 的 Multiplexier
module Mux4to1_1 (A, B, C, D, S1, S0, Out);
input  A, B, C, D, S1, S0;
output Out;
reg    Out;

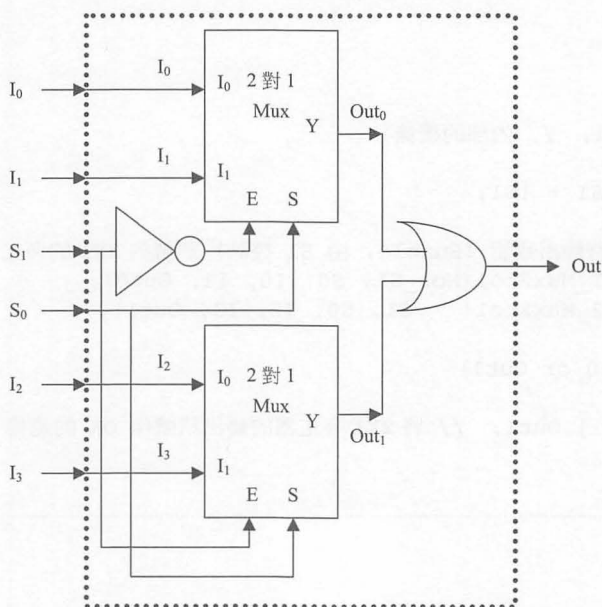
    always @(A or B or C or D or S1 or S0)
    begin
        case ({S1, S0})
            2'b00: Out = A;
            2'b01: Out = B;
            2'b10: Out = C;
            2'b11: Out = D;
        endcase
    end
endmodule
```



▲ Mux4_1_1.V 的模擬結果

(三) 使用二個 2X1 的多工器擴充成一個 4X1 的多工器

我們可以使用二個 2X1 的多工器擴充成一個 4X1 的多工器，如圖「使用二個 2X1 的多工器擴充成一個 4X1 的多工器的邏輯符號」所示。



▲ 使用二個 2X1 的多工器擴充成一個 4X1 的多工器的邏輯符號

當選擇線 (S_1) 為 0 時，經由反向器後訊為 1，因此選擇到了上面的那一個 2 對 1 Mux，輸出資料連到 OR 閘輸入端的 Out_0 ，由選擇線 S_1 及 S_0 來選擇輸入訊號 I_0 或 I_1 ，同時下面的那一個 2 對 1 Mux 不動作；因此對整個模組的輸出就等於 Out_0 。

反之，當選擇線 (S_1) 為 1 時，直接選擇到了下面的那一個 2 對 1 Mux，輸出資料連到 OR 閘輸入端的 Out_1 ，由選擇線 S_1 及 S_0 來選擇輸入訊號 I_2 或 I_3 ，同時上面的那一個 2 對 1 Mux 不動作，因此對整個模組的輸出就等於 Out_1 。

【Mux4_1b2.V：使用二個 2X1 的多工器擴充成一個 4X1 的多工器】

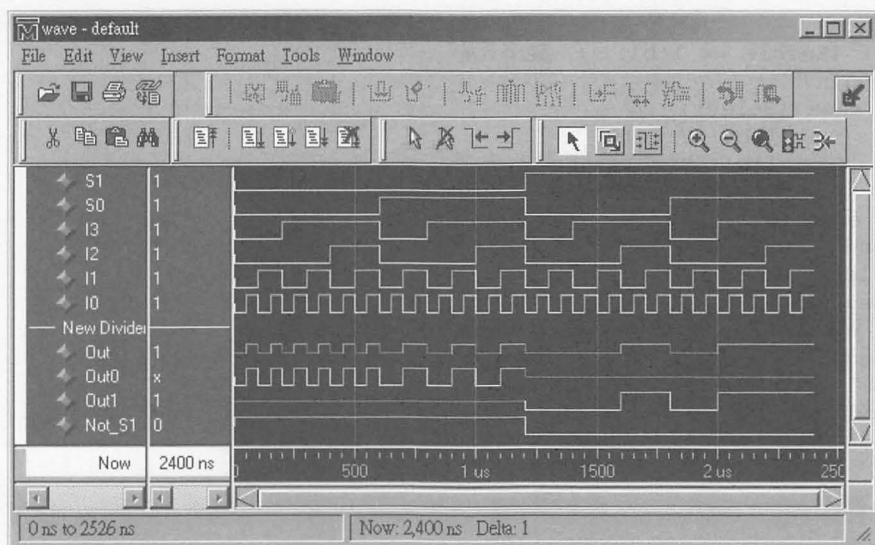
```
// Mux4_1b2.V：使用二個 2X1 的多工器擴充成一個 4X1 的多工器
// 引用 Mux2to1E.V，一個具有輸出致能 (Enable) 控制訊號的 2X1 的多工器
`include "Mux2to1E.V"

// 使用二個 2X1 的多工器擴充成一個 4X1 的多工器
module Mux4x1_By_Mux2to1E (S1, S0, I3, I2, I1, I0, Out);
input S1, S0, I3, I2, I1, I0;
output Out;
reg Out;
wire Out0;
wire Out1;
wire Not_S1; // 內部的接線

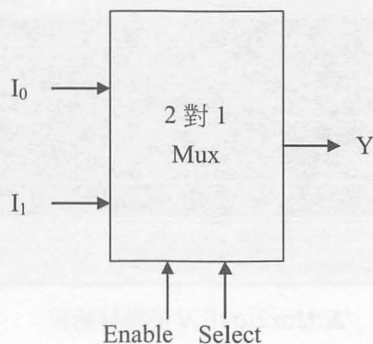
assign Not_S1 = !S1;

// 引用 2 個具有輸出致能 (Enable, 由 S1 控制) 訊號的 2X1 的多工器
Mux2to1E My1_Mux2to1(Not_S1, S0, I0, I1, Out0);
Mux2to1E My2_Mux2to1(S1, S0, I2, I3, Out1);

always @(Out0 or Out1)
begin
    Out = Out0 | Out1; // 將 2X1 多工器的輸出訊號作 OR 的連接
end
endmodule
```



▲ Mux4_1b2.V 的模擬結果



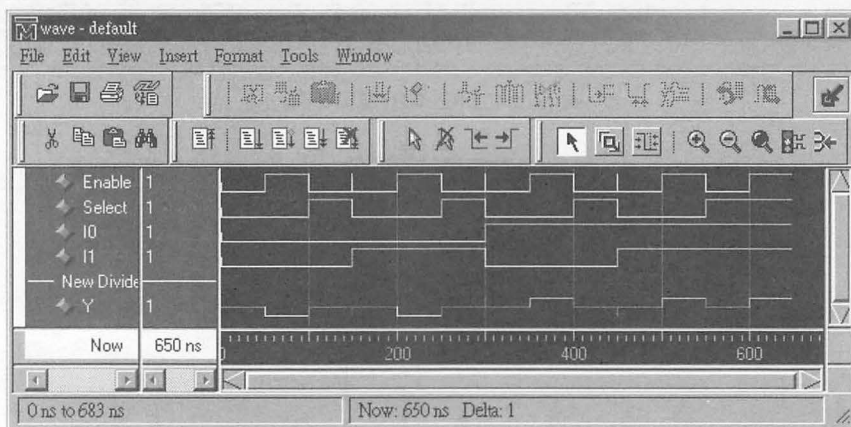
▲ Mux2to1E.V：具有輸出致能（Enable）控制訊號的 2X1 多工器的邏輯符號

【Mux2to1E.V：具有輸出致能(Enable)控制訊號的 2X1 的多工器】

```
// Mux2to1E.V：具有輸出致能（Enable）控制訊號的 2X1 的多工器
module Mux2to1E (Enable, Select, I0, I1, Y);
input  Enable, Select, I0, I1;
output Y;
reg    Y;

always @(Enable or Select or I0 or I1)
```

```
begin
  if (Enable == 1'b1) // 輸出致能
  begin
    case (Select)
      1'b0: Y = I0;
      1'b1: Y = I1;
      default: Y = 1'bx; // 不確定值, Unknow
    endcase
  end
else
  begin
    Y = 1'bx; // 不確定值, Unknow
  end
end
endmodule
```



▲ Mux2to1E.V 的模擬結果

8.1.5 解多工器 (DeMultiplexier)

解多工器的動作方式：從單一輸入線輸入值，然後傳送到 2^n 個可能輸出中的一條輸出線上。它的動作方式恰好跟多工器的相反。

典型的 1×2^n (1 對 2^n) 解多工器，它具有 n 條選擇線 ($S_{n-1}, S_{n-2}, \dots, 1, 0$) 以及 2^n 條輸出線 ($Y_{2^n-1}, Y_{2^n-2}, \dots, Y_1, Y_0$)。當解多工器動作後，這 2^n 條輸出線只有 1 條輸出線的值會等於單一輸入線的輸入值。

在本章節中，我們使用 Verilog 硬體描述語言的各種寫法來描述一個 1 對 2 的解多工器、一個 1 對 4 的解多工器，以及使用二個 1X2 的解多工器擴充成一個 1X4 的解多工器作為例子。

(一) 1 對 2 的解多工器

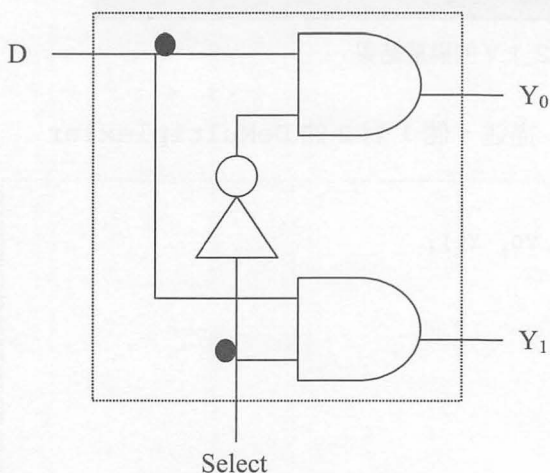
1 對 2 解多工器的真值表

Select	Y_0	Y_1
0	D	0
1	0	D
X	X	X

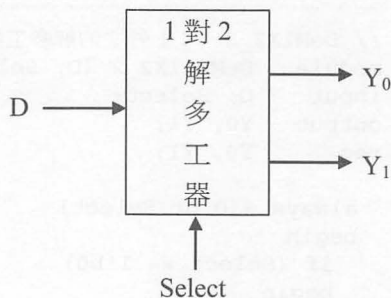
【1 對 2 解多工器的布林方程式】

$$Y_0 = \text{Select}' \cdot D$$

$$Y_1 = \text{Select} \cdot D$$



▲ 1 對 2 解多工器的邏輯電路圖

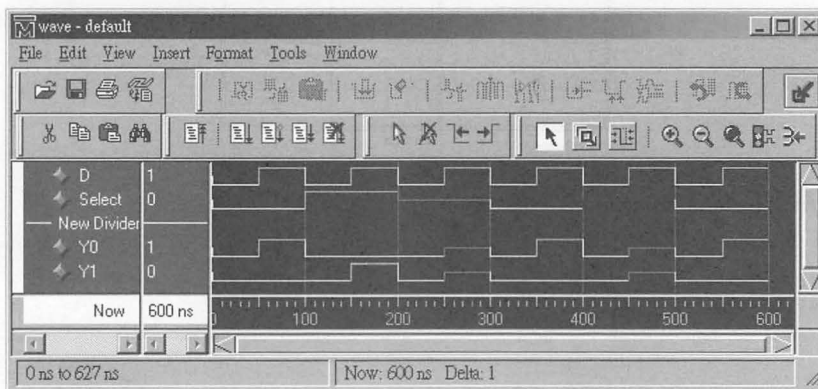


▲ 1 對 2 解多工器的邏輯符號

例 1 使用 assign 敘述，描述一個 1 對 2 的解多工器

```
// DeMux1X2_1.V : 1 對 2 的解多工器
module DeMux1X2_1 (D, Select, Y0, Y1);
input D, Select;
output Y0, Y1;
wire Y0, Y1;

    assign Y0 = (!Select) & D;
    assign Y1 = Select & D;
endmodule
```



▲ DeMux1X2_1.V 的模擬結果

例 2 使用 if ... else ... 敘述，描述一個 1 對 2 的 DeMultiplexier

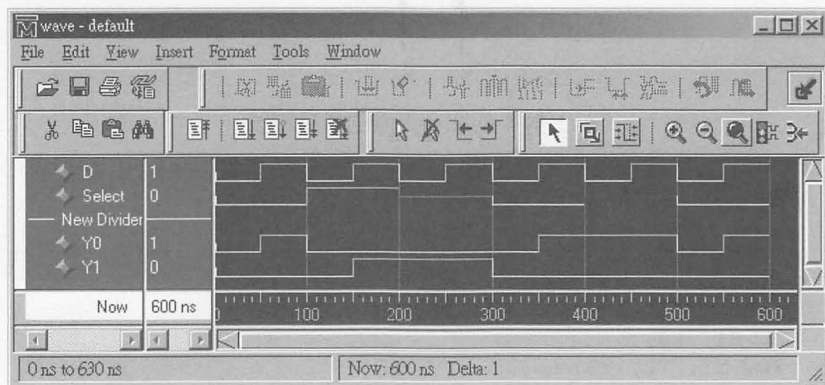
```
// DeMux1X2_2.V : 1 對 2 的解多工器
module DeMux1X2_2 (D, Select, Y0, Y1);
input D, Select;
output Y0, Y1;
reg Y0, Y1;

    always @(D or Select)
    begin
        if (Select == 1'b0)
        begin
            Y0 = D;
            Y1 = 0;
        end
        else
        begin
            Y0 = 0;
            Y1 = D;
        end
    end
endmodule
```

```

if (Select == 1'b1)
begin
    Y0 = 0;
    Y1 = D;
end
end
endmodule

```



▲ DeM1X2_2.V 的模擬結果

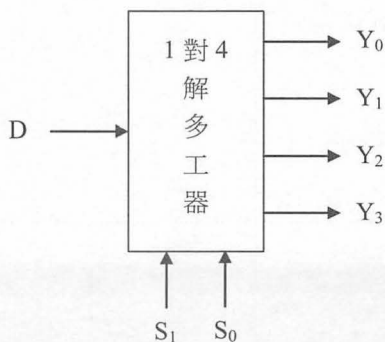
(二) 1 對 4 的解多工器

1 對 4 解多工器的真值表

S1	S0	Y ₀	Y ₁	Y ₂	Y ₃
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D
X	X	X	X	X	X

【1 對 4 的解多工器的布林方程式】

$$\begin{aligned}
 Y_0 &= S1' \cdot S0' \cdot D \\
 Y_1 &= S1' \cdot S0 \cdot D \\
 Y_2 &= S1 \cdot S0' \cdot D \\
 Y_3 &= S1 \cdot S0 \cdot D
 \end{aligned}$$

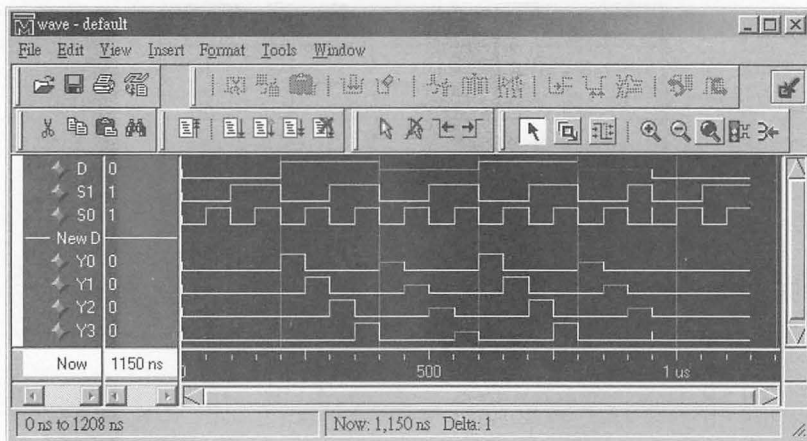


▲ 1 對 4 的解多工器的邏輯符號

例 1 使用 assign 敘述，描述一個 1 對 4 的解多工器

```
// DeMux1X4_1.V : 1 對 4 的解多工器
module DeMux1X4_1 (D, S1, S0, Y0, Y1, Y2, Y3);
input D, S1, S0;
output Y0, Y1, Y2, Y3;
wire Y0, Y1, Y2, Y3;

assign Y0 = (!S1) & (!S0) & D;
assign Y1 = (!S1) & S0 & D;
assign Y2 = S1 & (!S0) & D;
assign Y3 = S1 & S0 & D;
endmodule
```

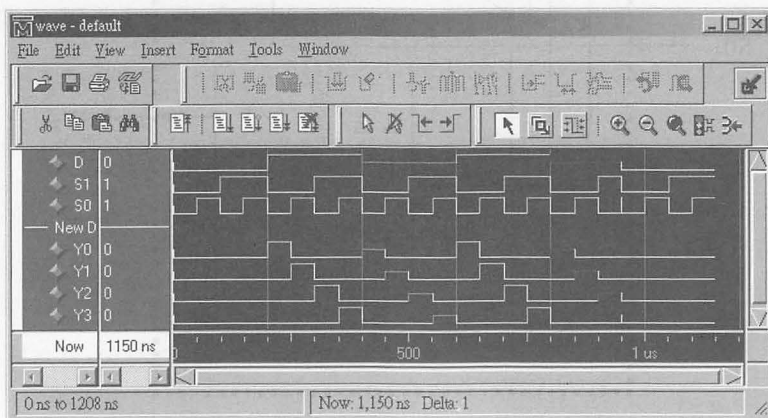


▲ DeMux1X4_1.V 的模擬結果

例 2 使用 if ... else ... 敘述，描述一個 1 對 4 的解多工器

```
// DeMux1X4_2.V: 1 對 4 的解多工器
module DeMux1X4_1 (D, S1, S0, Y0, Y1, Y2, Y3);
input D, S1, S0;
output Y0, Y1, Y2, Y3;
reg Y0, Y1, Y2, Y3;

always @(D or S0 or S1)
begin
    if ((S1 == 1'b0) & (S0 == 1'b0))
    begin
        Y0 = D; Y1 = 0; Y2 = 0; Y3 = 0;
    end
    else
    if ((S1 == 1'b0) & (S0 == 1'b1))
    begin
        Y0 = 0; Y1 = D; Y2 = 0; Y3 = 0;
    end
    else
    if ((S1 == 1'b1) & (S0 == 1'b0))
    begin
        Y0 = 0; Y1 = 0; Y2 = D; Y3 = 0;
    end
    else
    if ((S1 == 1'b1) & (S0 == 1'b1))
    begin
        Y0 = 0; Y1 = 0; Y2 = 0; Y3 = D;
    end
end
endmodule
```



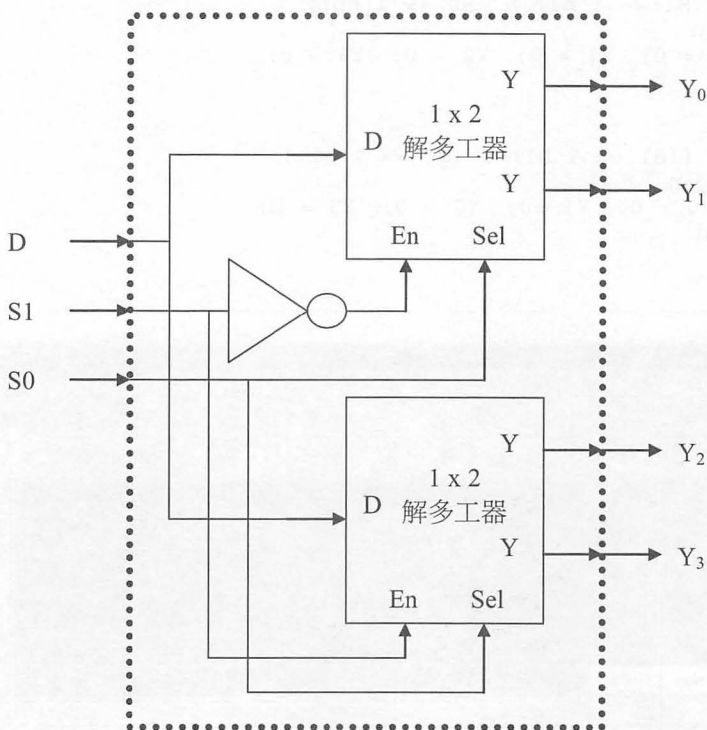
▲ DeMux1X4_2.V 的模擬結果

(三) 使用二個 1X2 的解多工器擴充成一個 1X4 的解多工器

我們可以使用二個 1X2 的解多工器擴充成一個 1X4 的解多工器，如圖「使用二個 1X2 的解多工器擴充成一個 1X4 的解多工器的邏輯符號」所示。

當選擇線 (S_1) 為 0 時，經由反向器後訊為 1，因此選擇到了上面的那一個 1 對 2 的解多工器，輸出資料 $Y_0 = D$ 或 $Y_1 = D$ ，由選擇線 S_0 來決定，同時下面的那一個 2 對 1 Mux 不動作，輸出資料 Y_2 和 Y_3 均等於 0。

反之，選擇線 (S_1) 為 1 時，直接選擇到下面的那一個 1 對 2 的解多工器，輸出資料 $Y_2 = D$ 或 $Y_3 = D$ ，由選擇線 S_0 來決定，同時上面的那一個 2 對 1 Mux 不動作，輸出資料 Y_0 和 Y_1 均等於 0。



▲ 使用二個 2X1 的多工器擴充成一個 4X1 的多工器的邏輯符號

【DeM1_4b2.V：使用二個 1X2 的解多工器擴充成一個 1X4 的解多工器】

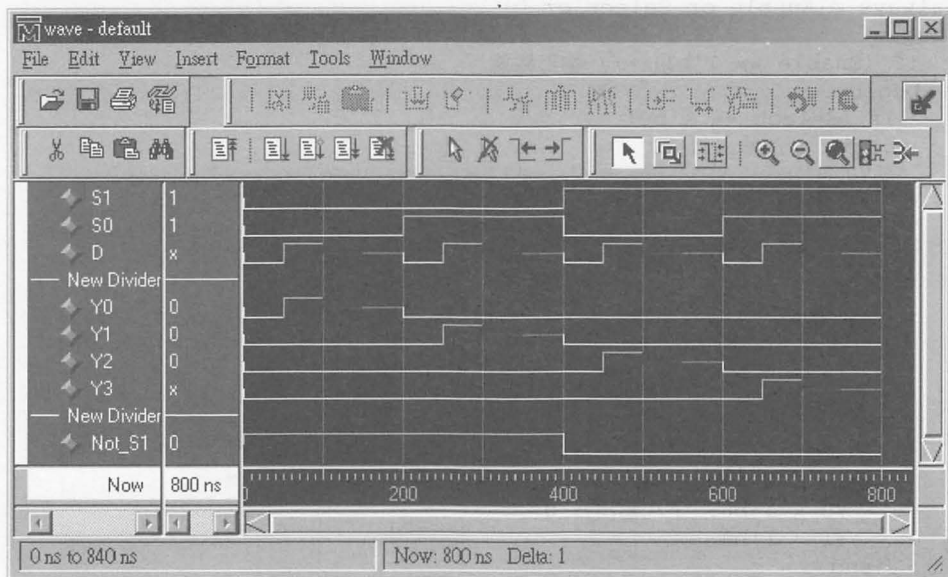
```
// DeM1_4b2.V：使用二個 1X2 的解多工器擴充成一個 1X4 的解多工器

// 引用 DeM1to2E.V
// 一個具有輸出致能 (Enable) 控制訊號的 1X2 的解多工器
`include "DeM1to2E.V"

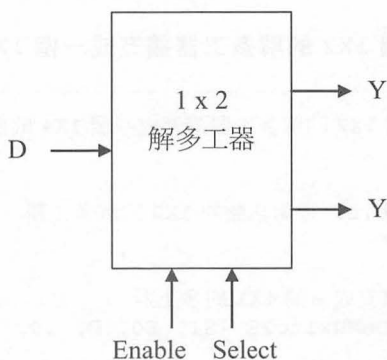
// 使用二個 2X1 的多工器擴充成一個 4X1 的多工器
module DeMux1x4_By_DeMux1to2E (S1, S0, D, Y0, Y1, Y2, Y3);
input S1, S0, D;
output Y0, Y1, Y2, Y3;
wire Y0, Y1, Y2, Y3;
wire Not_S1; // 內部的接線

assign Not_S1 = !S1;
// 引用 2 個具有輸出致能 (Enable) 控制訊號的 1X2 解多工器
DeMux1to2E My1_DeMux1to2E(Not_S1, S0, D, Y0, Y1);
DeMux1to2E My2_DeMux1to2E(S1, S0, D, Y2, Y3);

endmodule
```



▲ DeM1_4b2.V 的模擬結果



▲ DeM2to1E.V：具有輸出致能（Enable）控制訊號的 1X2 解多工器的邏輯符號

【DeM1to2E.V：具有輸出致能（Enable）控制訊號的 1X2 解多工器】

```

// DeM1to2E.V：具有輸出致能（Enable）控制訊號的 1X2 解多工器
module DeMux1to2E(Enable, Select, D, Y0, Y1);
input Enable, Select, D;
output Y0, Y1;
reg Y0, Y1;

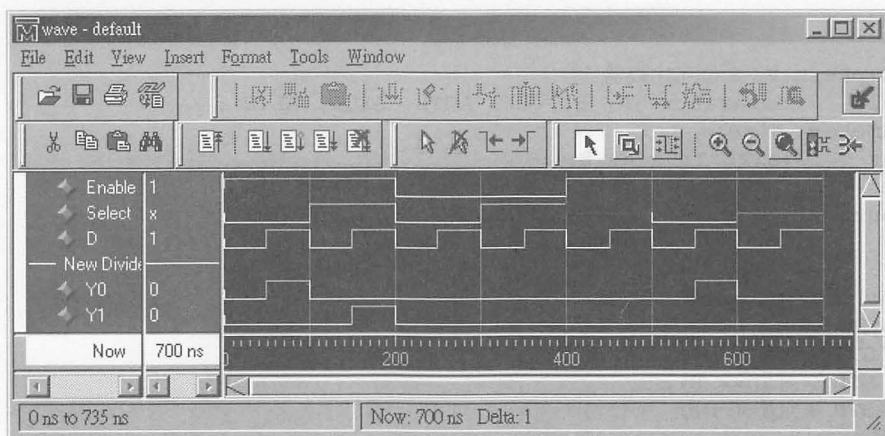
always @(Enable or Select or D)
begin
  if (Enable == 1'b1) // 輸出致能
  begin
    if (Select == 1'b0)
    begin // 輸出到 Y0
      Y0 = D;
      Y1 = 0;
    end
    else
    begin
      if (Select == 1'b1)
      begin // 輸出到 Y1
        Y0 = 0;
        Y1 = D;
      end
      else
      begin
        Y0 = 1'b0; // 輸出為 0
        Y1 = 1'b0;
      end
    end
  end
end
end
  
```

```

else
begin
    Y0 = 1'b0; // 輸出為 0
    Y1 = 1'b0;
end

end
endmodule

```



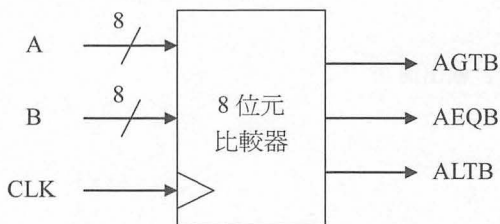
▲ DeM1to2E.V 的模擬結果

8.1.6 比較器 (Comparator)

數位電路中的比較器之設計，通常依據輸入的二組 2 進位數的數值大小來作比較。

二組 2 進位數的數值比較結果有三種情形： $A > B$ 、 $A = B$ 及 $A < B$ ，且這三種情況只有一個為真 (True)、其餘為偽 (False)。

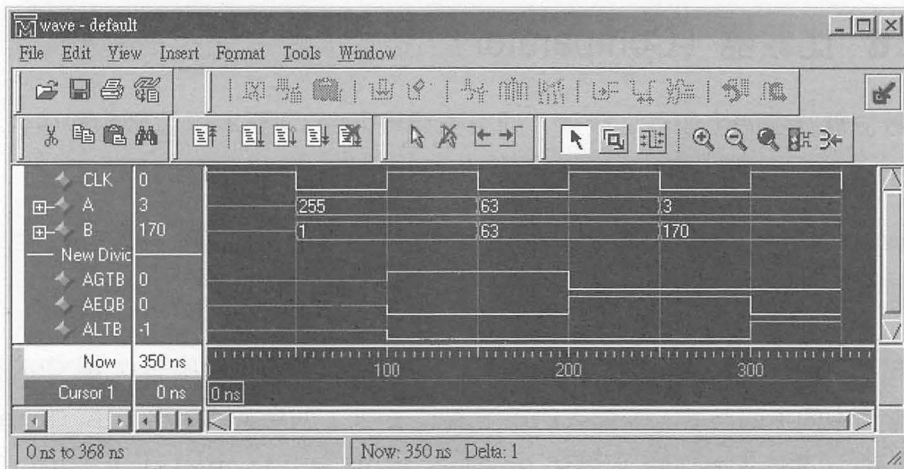
在本章節中，我們將設計一個 8 位元的比較器，該模組中有 1 個 CLK 訊號作為電路中同步的時脈，若輸入的訊號 A 比 B 大，則令 AGTB 為 1 (A Greater Than B)；若輸入的訊號 A 等於 B，則令 AEQB 為 1 (A Equal to B)；若輸入的訊號 A 比 B 小，則令 ALTB 為 1 (A Less Than B)。



▲ 8 位元比較器 (Comparator)

```
// Cmp8Bit.V: 8 位元比較器
module    Comparator (A, B, CLK, AGTB, AEQB, ALTB);
parameter size = 8;
input     [size-1:0] A, B;
input     CLK;
output    AGTB, AEQB, ALTB;
reg       AGTB, AEQB, ALTB;

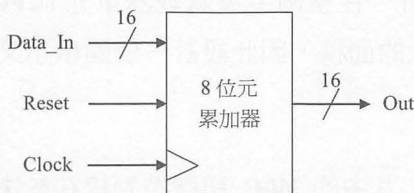
always @(posedge CLK)
begin
    AGTB = (A > B);
    AEQB = (A == B);
    ALTB = (A < B);
end
endmodule
```



▲ Cmp8Bit.V 的模擬結果

8.1.7 累加器 (Accumulator)

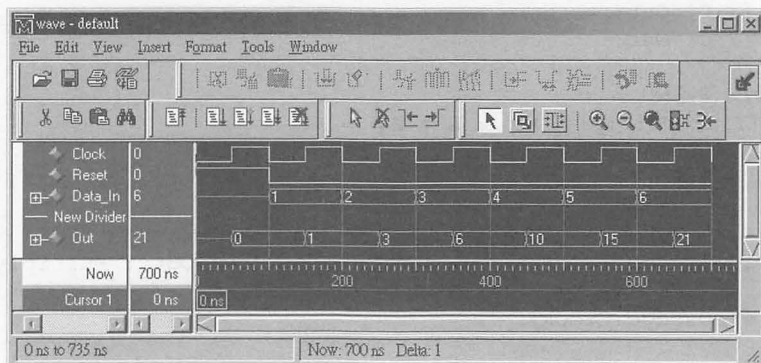
累加器的功能在於讓電路中的某個暫存器值能夠一直累加特定的值而稱之。



▲ 8 位元累加器

```
// Accumu.V : 8 位元累加器
// 具有「重置」訊號控制的累加器
module Accumu (Clock, Reset, Data_In, Out);
input      Clock, Reset;
input [15:0] Data_In;
output [15:0] Out;
reg [15:0] Out;

always @(posedge Clock)
begin
    if (Reset) // 重置
        Out = 0; // 清除為 0
    else // 從輸入埠 Data_In 讀入累加值，輸出埠 Out 加上 Data_In
        Out = Out + Data_In;
    end
endmodule
```

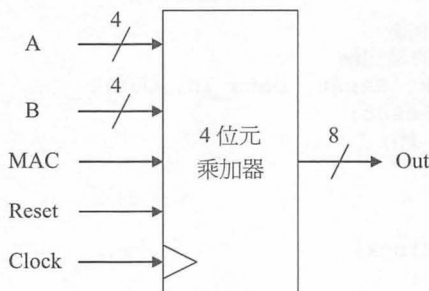


▲ Accumu.V 的模擬結果

8.1.8 乘加器 (Multiplier and Accumulator)

乘加器簡單的說就是結合了乘法器與加法器的功能，它是屬於算數邏輯運算單元 (ALU) 中的一部份，在整個中央處理器單元 (CPU) 中佔有舉足輕重的地位，原因是它佔了相當大的面積。因此設計一個面積小又高效能的乘加器是一門很大的學問。

例 1 4 位元乘加器，其中的 MAC 訊號控制線在於決定累加值為：將原來內部的累加值再加上 $A * B$ 的值，或者是原來內部的累加值不變動。



```

// Mac1.V: 乘加器, 範例 1
// 具有「重置」及「乘加選擇」訊號控制的乘加器
module Mac1 (Clock, Reset, MAC, A, B, Out);
input      Clock, Reset, MAC;
input      [3:0] A, B;
output     [7:0] Out;
reg        [7:0] Out;

// 宣告一個乘法處理
wire       [7:0] Multiply_Out = A * B;

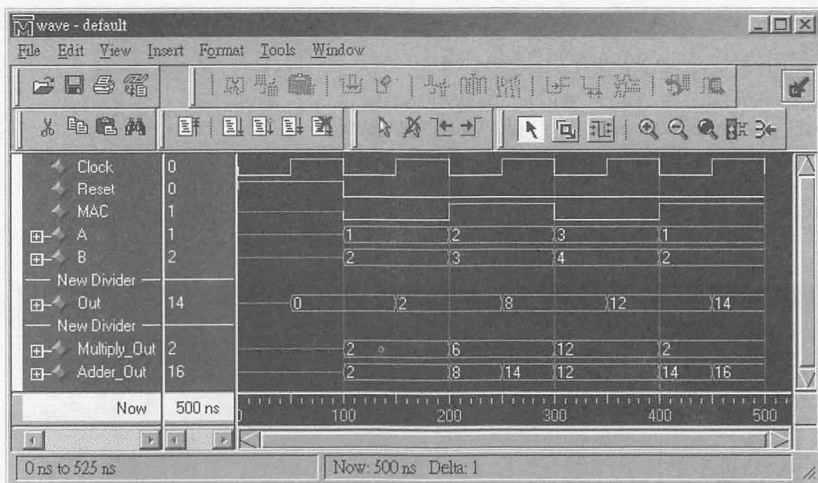
// 宣告一個乘加處理：
// 若 MAC = 1, 則 Adder_Out = Multiply_Out + Out,
// 否則 Adder_Out = Multiply_Out
wire       [7:0] Adder_Out = MAC ? Multiply_Out + Out : Multiply_Out;

always @(posedge Clock)
begin
    if (Reset) // 重置
        Out = 0; // 清除為 0
  
```

```

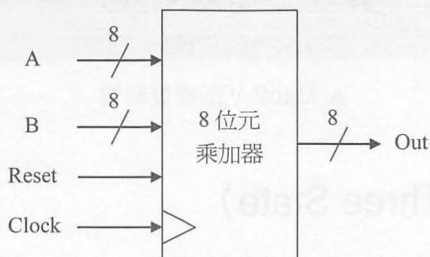
else
    Out = Adder_Out; // 輸出運算結果
end
endmodule

```



▲ Mac1.V 的模擬結果

例 2 8 位元乘加器，少了 MAC 訊號控制線，也就是會一直累加 $A \cdot B$ 的值。



```

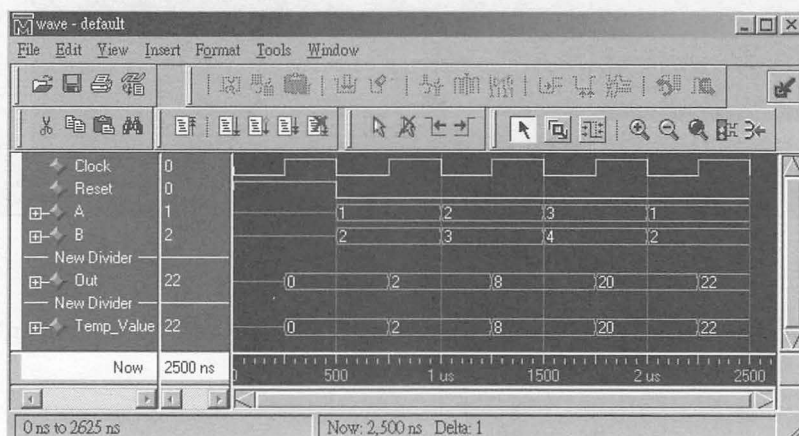
// Mac2.V：乘加器，範例 2
// 具有「重置」訊號控制的乘加器
module Mac2 (Clock, Reset, A, B, Out);
    in'put      Clock, Reset;
    input       [7:0] A, B;
    output      [7:0] Out;
    reg         [7:0] Out, Temp_Value;

```

```

always @(posedge Clock) // 正緣觸發重置訊號
begin
    if (Reset)
    begin
        Temp_Value = 8'd0; // 清除暫存結果的暫存器為 0
        Out = 8'd0; // 清除輸出暫存器為 0
    end
    else
    begin
        Temp_Value = (A * B) + Temp_Value; // 乘加計算
        Out = Temp_Value; // 儲存運算完的結果
    end
end
endmodule

```

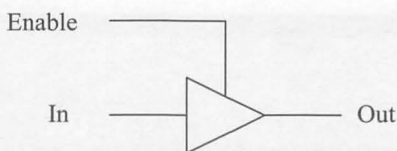


▲ Mac2.V 的模擬結果

8.1.9 三態閘 (Three State)

在 Verilog 硬體描述語言中，所定義的資料型態中有一種值「z」是用來表示高阻抗輸出的。而一般電路中所謂的三態輸出，就是指邏輯值除了有「0」與「1」以外，還有另外二種電路的訊號：「z」【高阻抗(High Impedance)、浮接狀態(Floating State)、Tri-States、Disabled Driver(Unknown)】及「x」【不確定值(Unknown Value)】(註：我們在「Verilog 的資料型態(Data Types)」這節中有提到)。

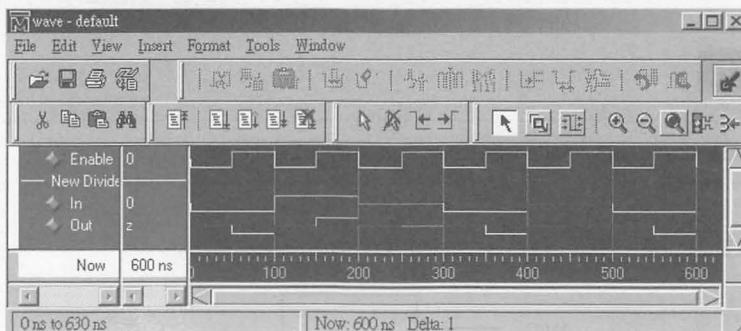
三態閘 (Three State) 的動作方式為：如果 Enable 端為 1，則 $Out = In$ ，否則 Out 為 x (不確定值，Unknown Value)。



(一) 三態閘 (Tri-State) 輸出驅動，範例 1

```
// TriStat1.V: 三態閘 (Tri-State) 輸出驅動範例 1
module TriState_1 (Enable, In, Out);
input Enable, In;
output Out;
wire Out;

assign Out = Enable ? In : 'bz;
endmodule
```

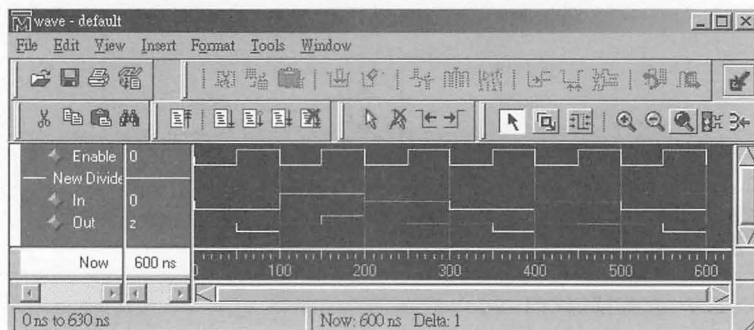


▲ TriStat1.V 的模擬結果

(二) 三態閘 (Tri-State) 輸出驅動，範例 2

```
// TriStat2.V: 三態閘 (Tri-State) 輸出驅動範例 2
module TriState_2 (Enable, In, Out);
input Enable, In;
output Out;
wire Out;
```

```
    bufif1(Out, In, Enable);
endmodule
```



▲ TriStat2.V 的模擬結果

8.1.10 雙向埠 (Bidirectional Port, 使用 inout 敘述)

使用 inout 敘述所定義的埠，既有輸入埠的特性也有輸出埠的功能。

BNF 表示法

```
<inout_declaration> := inout <range>? <list_of_variables>;
```

範例

BiDir.V : 雙向 (Bidirectional) 驅動範例



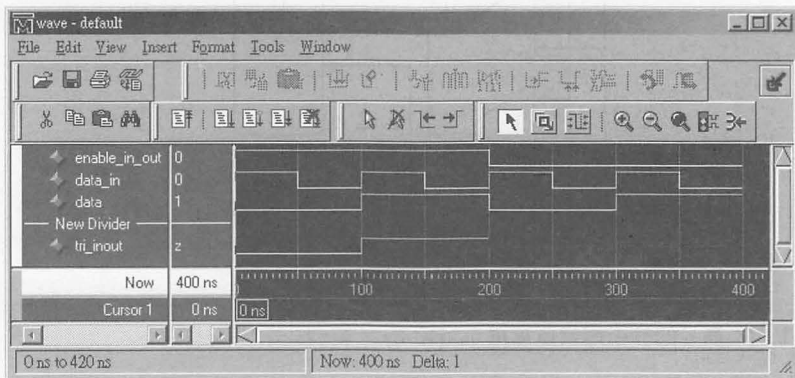
```
// BiDir.V: Bidirectional 驅動範例
// 如果 enable_in_out = 1 且 data_in = 1, 則 tri_inout = data
// 如果 enable_in_out = 1 且 data_in = 0, 則 tri_inout 的值不變
// 如果 enable_in_out = 0, 則 tri_inout = 1'bz
module BiDir (enable_in_out, data_in, data, tri_inout);
```

```

input    enable_in_out, data_in, data;
inout    tri_inout;
wire     tri_inout;

assign tri_inout = enable_in_out ? ((data_in) ? data :
tri_inout) : 1'bz;
endmodule

```



▲ BiDir.V 的模擬結果

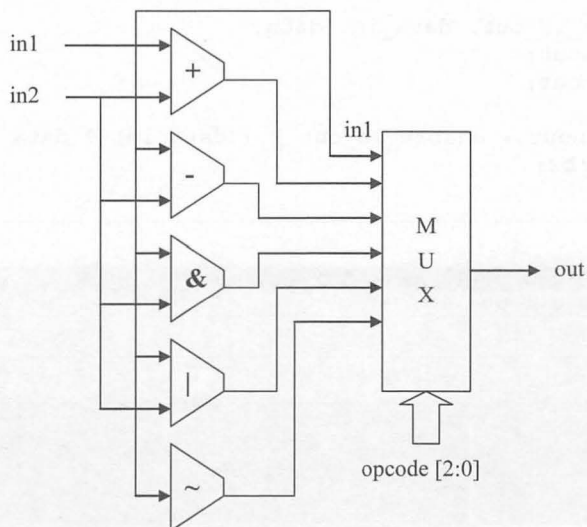


8.2 簡易的算術邏輯運算單元 (ALU) 的設計

算術邏輯運算單元 (ALU) 為結合了數位系統中最基本的：加法 (+)、減法 (-)、AND (&)、OR (|)、Not (!)...等等的運算，由運算碼 (operator) 來決定您想對輸入的資料作那一種的運算。

範例 1 ALU_1.V，簡易的算術邏輯運算單元 (ALU)

在本範例中：運算碼 (operator) 可以是加法、減法、AND、OR、Not 等其中一種。只要是輸入的資料或者是運算碼 (operator) 有任何的改變後，就會重新計算。



// ALU_1.V：簡易的算術邏輯運算單元 (ALU)，範例

```

`define Plus      3'd0
`define Minus     3'd1
`define Bin_And   3'd2
`define Bin_Or    3'd3
`define Unegate   3'd4

module ALU_1 (opcode, in1, in2, out);
input      [2:0] opcode;
input      [7:0] in1, in2;
output     [7:0] out;
reg        [7:0] out;

always @(opcode or in1 or in2)
begin
    case (opcode)
        // Arithmetic operations
        `Plus:
            out = in1 + in2;
        `Minus:
            out = in1 - in2;

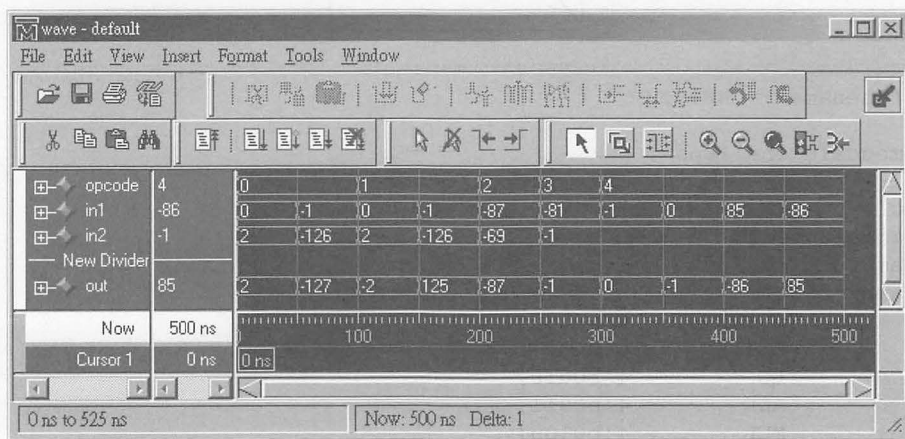
        // Bitwise operations
        `Bin_And:
            out = in1 & in2;
        `Bin_Or:
            out = in1 | in2;
    endcase
end

```



```
// Unary operations
`Unegate:
    out = ~in1;

default:
    out = 8'hx;
endcase
end
endmodule
```



▲ ALU_1.V 的模擬結果

範例 2 CHU96.V，簡易的算術邏輯運算單元 (ALU) 的第 1 種寫法

在本範例中：運算碼 (operator) 可以是 CLEAR (清除 ALU 內的運算結果)、CLEARC (如果 Carry 旗標為 1，則清除 ALU 內的運算結果)、LD (載入第一個資料輸入埠的值到 ALU 內的結果變數中)、ADD (加法動作)、MUL (乘法動作)、SHL (將 ALU 內的結果變數值向左移一個位元，相當於一個乘 2 的動作)、ROL (將 ALU 內的結果變數值向左旋轉一個位元，最高的位元則是移到最低的位元裡) 等其中一種。只要是輸入的資料或者是運算碼 (operator) 有任何的改變後，就會重新計算，並由運算完的結果計算出 Carry (進位) 判斷、Even (偶數) 判斷、Parity (奇同位元) 判斷、Zero (零值) 判斷及 Negative (正負數) 判斷等常見的旗標 (flags) 訊號。



```
// filename : CHU96.V
```

```
`timescale 1ns/10ps
```

```
module      CHU96 (clock, in1, in2, op, out, c, e, p, z, n);
parameter   Bits      = 4;
parameter   Ops       = 4;
parameter   [Ops-1:0] // synopsys enum operator
    CLEAR      = 4'b0000,
    CLEARC     = 4'b0001,
    LD         = 4'b0010,
    ADD        = 4'b0011,
    MUL        = 4'b0100,
    CMP        = 4'b0101,
    SHL        = 4'b0110,
    ROL        = 4'b0111;

input       clock;
input       [Bits-1:0] in1;
input       [Bits-1:0] in2;
input       [Ops-1:0] op;
reg         [2*Bits :0] alu;
output      [2*Bits-1:0] out;
wire        [2*Bits-1:0] out;
output      c; // Carry
wire        c;
output      e; // Even
wire        e;
output      p; // Parity
wire        p;
output      z; // Zero
wire        z;
output      n; // Negative
wire        n;
```

```
assign          c = alu[2*Bits];
assign          e = ~alu[0];
assign          p = ^alu;
assign          z = ~alu; // ~ : 1's Complement
assign          n = alu[2*Bits-1];
assign out[2*Bits-1:0] = alu[2*Bits-1:0];

always @(posedge clock)
begin
    case(op)          // synopsys parallel_case
        CLEAR :
            alu = 0;

        CLEARC :
            if(c == 0)
                alu = 0;

        LD :
            alu = in1;

        ADD :
            alu = in1 + in2;

        MUL :
            alu = in1 * in2;

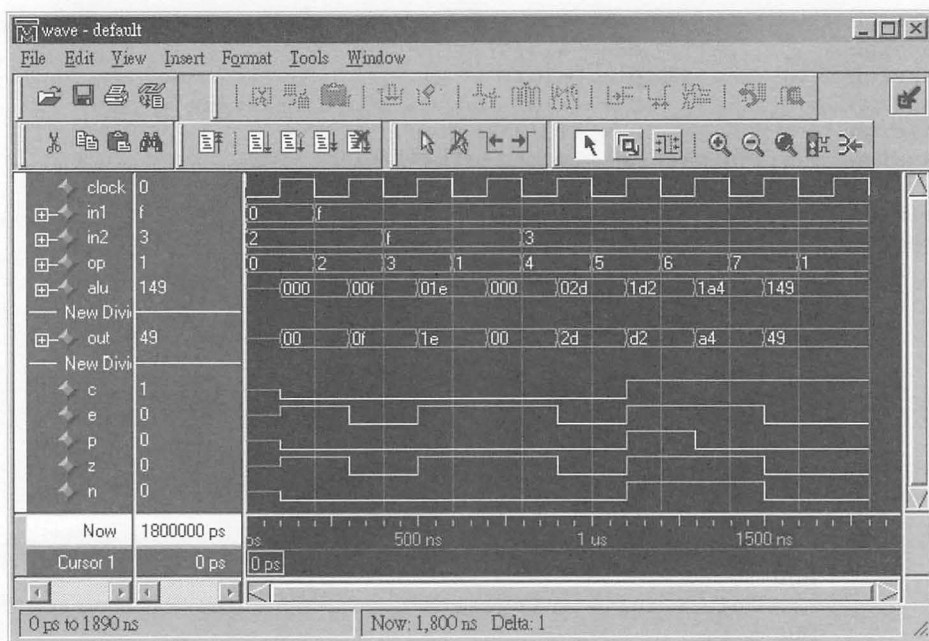
        CMP : // 1's Complement
            alu = ~out;

        SHL :
            alu = out << 1;

        ROL :
            begin
                alu = out << 1;
                alu[0] = alu[2*Bits];
            end

    endcase
end

endmodule
```

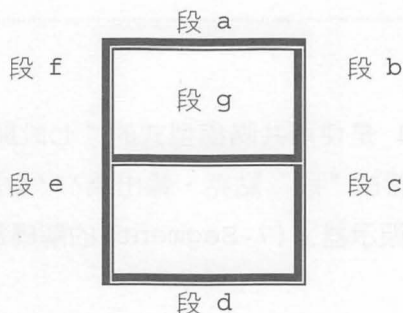


▲ CHU96.V 的模擬結果

本章練習

Question 1 :

請設計共陰極型式『七段顯示器』(7 Segment)的解碼電路。



七段顯示器

假設要使用共陰極型式的“七段顯示器”(7 Segment)，那麼輸出為‘1’會讓該“段”點亮、輸出為‘0’會讓該“段”不亮。

十進制值 二進制值 Data	0 0000	1 0001	2 0010	3 0011	4 0100	5 0101	6 0110	7 0111	8 1000	9 1001
段 a	●		●	●		●	●	●	●	●
段 b	●	●	●	●	●			●	●	●
段 c	●	●		●	●	●	●	●	●	●
段 d	●		●	●		●	●		●	●
段 e	●		●				●		●	
段 f	●				●	●	●		●	●
段 g			●	●	●	●	●		●	●

● 代表：亮，空格代表：不亮。

可以用“卡諾圖”(Karnaugh)來化簡，段 a~段 g 輸出的方程式。

```
module Segment_7 (Data, a, b, c, d, e, f, g);
input  [3:0] Data;
output  a, b, c, d, e, f, g;
wire    a, b, c, d, e, f, g;

    assign a = ?    assign b = ?    assign c = ?    assign d = ?
    assign e = ?    assign f = ?    assign g = ?
endmodule
```

Question 2 :

如果 Question 1 是使用共陽極型式的“七段顯示器”(7 Segment)，那麼輸出為‘0’會讓該“段”點亮、輸出為‘1’會讓該“段”不亮，請設計共陽極型式『七段顯示器』(7 Segment)的解碼電路。

Question 3 :

請設計一個能做『整數』加法、減法、乘法、除法以及取餘數的電路。

Question 4 :

請設計一個能求出二個整數的最大公因數(Greatest Common Divisor，簡寫為 G.C.D.或 Highest Common Factor，簡寫為 H.C.F.)。

Question 5 :

請設計一個能求出二個整數的最小公倍數(Least Common Multiple，簡寫為 L.C.M.)。

PS Google 可以幫您找到解答。